



COMP6841 Security Engineering Project Report

VulnerableNotes

Build, Demonstrate, Fix, and Analyse Common Web Vulnerabilities

Course: COMP6841 Security Engineering, UNSW

Student: Zack Zhou (z5602186)

Date: July 2026

Github repo: <https://github.com/StickyGrenade/comp6841-web-security-project>

Abstract

This project is a deliberately vulnerable local notes application called VulnerableNotes. I built a small Flask and SQLite web app, intentionally introduced four common web vulnerabilities, demonstrated each one safely on my own machine, then patched the same issues in a separate secure version called SecureNotes. The four issues were SQL injection in note search, stored cross-site scripting (XSS) in note rendering, broken access control / IDOR on note detail pages, and plaintext password storage. The goal was not to build a production notes app. The goal was to show how ordinary application code can create security failures, how those failures can be demonstrated in a controlled local setting, and how relatively small design changes can fix them. All testing was performed only against my own local application.

Table of Contents:

Abstract.....	2
Table of Contents:.....	3
1. Introduction.....	5
Context and scope.....	5
Problem statement.....	5
Comparison with the original proposal.....	5
2. Background.....	6
Web application security concepts.....	6
Related research and guidance.....	6
Why a notes app.....	7
3. Methodology and Implementation.....	7
Method.....	7
Technical design.....	7
Vulnerable version implementation:.....	8
SQL injection.....	8
Stored XSS.....	8
Broken access control / IDOR.....	8
Weak password handling.....	8
Fixed version implementation:.....	9
Demonstrations and evidence.....	9
4. Experimental Evaluation and Results.....	9
SQL injection.....	9
Stored XSS.....	10
Broken access control / IDOR.....	10
Weak password handling.....	10
Summary of outcomes against the proposal.....	10
5. Discussion and Reflection.....	11
What the results actually show.....	11
Ethical and professional issues.....	11
Strengths.....	12
Weaknesses.....	12
Challenges and how I changed.....	12
What I would do differently.....	13
6. Conclusion.....	13
7. References.....	14
8. Appendix.....	14
A. How to run the apps.....	14
B. Screenshot index.....	14
C. Vulnerability writeups.....	15

D. Key code comparisons.....	15
E. Progress evidence.....	15

1. Introduction

Context and scope

Web applications sit between users and sensitive data. Even a simple notes app has authentication, database queries, and HTML rendering. Each of those layers can fail if user input or access control is handled carelessly.

For COMP6841, I chose a practical “build, break, fix” project. I wanted something small enough that I could explain the whole system in a short presentation, but still technical enough to show real security engineering ideas from the course.

The project has two apps:

- `vulnerable_version/` — VulnerableNotes, the intentionally unsafe app
- `fixed_version/` — SecureNotes, the patched version

Both apps support registration, login, logout, creating notes, listing notes, viewing a note by ID, and searching notes. The vulnerable version runs on `http://127.0.0.1:5000/`. The fixed version runs on `http://127.0.0.1:5001/` so I can compare them side by side.

Problem statement

Many people learn vulnerabilities as abstract names from a list. I wanted to understand them as concrete mistakes in code I had written myself.

The problem this project addresses is:

How do common web vulnerabilities appear in ordinary application features, what impact do they have in a notes app, and what secure design choices prevent them?

I limited the scope to four issues that map cleanly to features in the app:

1. SQL injection in search
2. Stored XSS in note body rendering
3. Broken access control / IDOR on `/note/<id>`
4. Weak password handling through plaintext storage

I also set a hard ethics boundary. This project is local-only. I did not test these techniques against third-party systems, public websites, or accounts I do not own.

Comparison with the original proposal

My original proposal was to build a deliberately vulnerable web app, demonstrate selected vulnerabilities safely, fix them, and document the before-and-after lessons. That is what I completed.

Compared with the proposal, I kept the tech stack simple (Flask, SQLite, Jinja2) and avoided overbuilding. I completed all four planned vulnerability demonstrations and a separate fixed version. The main change from early planning was focusing on a notes app rather than a larger feature set, so each vulnerability had a clear place to appear and a clear fix to compare against.

2. Background

Web application security concepts

This project sits in the area of web application security. The OWASP Top 10 is a widely used overview of the most common risks in web apps [1]. Several of the issues I implemented map directly to that list:

- Injection, including SQL injection
- Cross-site scripting
- Broken access control
- Cryptographic failures / insecure credential storage

These are not exotic attack chains. They often come from ordinary decisions such as concatenating user input into SQL, turning off template escaping, checking login but not ownership, or storing passwords as plain text.

Related research and guidance

I used public security guidance rather than inventing my own definitions:

- OWASP material on SQL injection and parameterized queries [2]
- OWASP XSS prevention guidance, especially the idea that untrusted data should be escaped by default [3]
- OWASP access control guidance, which stresses that authentication and authorization are different [4]
- Common password storage advice: store salted password hashes, not the password itself [5]

Existing tools such as Burp Suite, OWASP ZAP, or sqlmap are useful for finding vulnerabilities in real targets. I deliberately did not centre the project on those tools. My aim was to understand the vulnerable code itself, demonstrate impact in a controlled app I owned, and write the secure alternative. That made the learning more about secure design than about operating a scanner.

Why a notes app

A notes app is a good teaching target because it has:

- private user data
- searchable text fields
- rendered HTML output
- object IDs in URLs
- password-based accounts

Those features create natural places for the four vulnerabilities without needing a large codebase.

3. Methodology and Implementation

Method

I built the project in milestones:

1. Flask scaffold
2. Registration and login
3. Notes features
4. SQL injection demonstration
5. Stored XSS demonstration
6. IDOR demonstration
7. Weak password handling demonstration
8. Fixed secure version
9. Final report and documentation

The method was intentional. I first built a working “normal” app, then introduced one vulnerability at a time. That made it easier to see what changed and why the insecure behaviour appeared. After the vulnerable version was complete, I mirrored the app into `fixed_version/` and patched each issue with a targeted fix.

I marked intentional insecure code with `VULNERABLE:` comments in the vulnerable version, and matching `FIXED:` comments in the secure version. That was useful for demos and for comparing the two files side by side.

Technical design

Tech stack:

- Python Flask
- SQLite

- Jinja2 templates
- Simple HTML/CSS
- Werkzeug password helpers in the fixed version

Main project layout:

```
vulnerable_version/ intentionally vulnerable app
fixed_version/      secure patched app
docs/               progress log, writeups, screenshots, this report
```

Both apps use the same core schema: a `users` table and a `notes` table. Notes belong to users through `user_id`.

Vulnerable version implementation:

SQL injection

In the vulnerable search route, user input is inserted directly into the SQL string:

```
unsafe_sql = (
    "SELECT id, title, body, created_at "
    "FROM notes "
    f"WHERE user_id = {session['user_id']} "
    f"AND (title LIKE '%{query}%' OR body LIKE '%{query}%') "
    "ORDER BY created_at DESC"
)
```

This is the classic failure: the database receives user input as part of the command, not only as data.

Stored XSS

On the note detail page, the body is rendered with Jinja's `| safe` filter:

```
{{ note["body"] | safe }}
```

That turns off automatic escaping, so stored HTML or JavaScript can run when the note is viewed.

Broken access control / IDOR

The vulnerable note detail route checks login, but looks up the note by ID only:

```
WHERE id = ?
```

Any logged-in user can open another user's note by changing the ID in the URL.

Weak password handling

Registration stores the password string directly, and login compares plaintext strings. If the SQLite file is leaked, every password is readable.

Fixed version implementation:

SecureNotes keeps the same features and roughly the same structure, but changes the unsafe decisions:

Issue	Vulnerable behaviour	Fixed behaviour
SQL injection	string-built search SQL	parameterized ? placeholders
Stored XSS	<code>safe</code> on note body	normal Jinja escaping
IDOR	lookup by note ID only	lookup by note ID and owning user
Passwords	plaintext insert/compare	Werkzeug hash and verify

Example fixed search query:

```
results = get_db().execute(
    """
    SELECT id, title, body, created_at
    FROM notes
    WHERE user_id = ?
    AND (title LIKE ? OR body LIKE ?)
    ORDER BY created_at DESC
    """,
    (session["user_id"], f"%{query}%", f"%{query}%"),
).fetchall()
```

Demonstrations and evidence

I captured screenshots throughout development under `docs/screenshots/`. Detailed per-vulnerability writeups are in `docs/vulnerability_writeups/`. Selected screenshots and code comparisons are also included in the Appendix.

4. Experimental Evaluation and Results

All demonstrations used fake local accounts only, for example `alice` and `bob`.

SQL injection

Setup: Alice and Bob each created a private note. I logged in as Bob.

Normal result: Searching for Bob's note text returned only Bob's matching note.

Vulnerable result: Searching with:

```
) OR 1=1 --
```

changed the query logic and returned unexpected results, including notes that a normal search should not return.

Fixed result: The same input was treated as ordinary search text. The fixed search did not return Alice's note through that payload.

Finding: One unsafe string construction was enough to break the intended search boundary. The fix was small, but the behavioural difference was obvious.

Stored XSS

Setup: I created a note with this body:

```
<script>alert('Stored XSS')</script>
```

Vulnerable result: Opening the note detail page triggered a browser alert. On the notes list page, the same content stayed escaped and did not run.

Fixed result: The detail page showed the payload as escaped text. No alert appeared.

Finding: The danger was not storing the note. The danger was rendering stored content as trusted HTML. Keeping escaping on by default was enough to stop this demo.

Broken access control / IDOR

Setup: Alice created a private note. Bob created his own note.

Vulnerable result: While logged in as Bob, changing the URL to Alice's note ID displayed Alice's note. Logged-out visitors were still redirected to login.

Fixed result: Bob received a 404 when requesting Alice's note ID. Bob could still open his own notes.

Finding: Authentication answered "who are you?" Authorization answered "are you allowed to see this?" The vulnerable app only answered the first question.

Weak password handling

Setup: I registered a fake account and inspected the local SQLite database.

Vulnerable result: The password column contained readable plaintext. Login still worked because the app compared plaintext strings.

Fixed result: The password column contained a Werkzeug/scrypt hash. Login still worked through hash verification, but the original password was not stored directly.

Finding: Plaintext storage makes a database leak immediately useful to an attacker. Hashing does not remove all risk, but it removes the most obvious failure mode.

Summary of outcomes against the proposal

Planned outcome	Achieved?	Evidence
Build a simple notes app	Yes	Working vulnerable and fixed apps

Demonstrate selected vulnerabilities locally	Yes	Screenshots and writeups for all four issues
Patch the issues in a secure version	Yes	SecureNotes on port 5001
Document before/after behaviour	Yes	Progress log, writeups, this report
Keep ethics local-only	Yes	All tests against 127.0.0.1 only

Overall, the project completed what the proposal set out to do. The main limitation is depth of background research rather than missing implementation.

5. Discussion and Reflection

What the results actually show

The most useful lesson for me was that these vulnerabilities were not complicated. SQL injection was one f-string. XSS was one `| safe`. IDOR was one missing `user_id` check. Weak passwords were one plaintext insert.

That changed how I think about secure software. Security engineering is not only about adding tools at the end. It is about ordinary design choices while writing normal features.

Another result that was less obvious at first is that “login required” can create a false sense of safety. The IDOR demo was the clearest example. The app looked protected because strangers could not view notes while logged out. Once Bob was logged in, ownership stopped mattering. That distinction between authentication and authorization is something I understand much more concretely now.

Ethical and professional issues

I treated ethics as part of the design, not an afterthought.

Choices I made:

- only test against my own local app
- use fake accounts and fake passwords
- keep demonstration payloads harmless
- mark intentional vulnerabilities clearly in code
- write documentation that explains impact without framing the project as an attack guide for real systems

This mattered because the same techniques are dangerous outside a controlled lab. Keeping the project local forced me to focus on understanding and fixing the issues rather than chasing flashy exploit outcomes.

Strengths

- Clear security engineering loop: build, demonstrate, fix, compare
- Each vulnerability has a clear feature location and a clear fix
- Side-by-side vulnerable and fixed apps make the before/after story easy to show
- Evidence exists as screenshots, writeups, and marked code

Weaknesses

- Background research could go deeper into related academic work, not only OWASP guidance
- The app is intentionally tiny, so it does not explore more advanced issues such as CSRF, session fixation, or rate limiting
- Some screenshots are evidence of behaviour, but a longer recorded demo would make the evaluation section stronger for an external reader

Challenges and how I changed

The first real challenge was learning Flask, Jinja, sessions, and SQLite at the same time. Early on I could write pieces of the app without fully understanding how they connected. Building in milestones forced me to slow down.

Another challenge was restraint. It was tempting to add more features or more vulnerabilities. Keeping the project explainable was harder than making it larger. That restraint turned out to be useful for both the report and the presentation.

I also found documentation harder than I expected. Writing past-tense public docs that show completed work is different from keeping private testing notes. I had to rewrite material so it read as my own record of what I built and learned, rather than as a checklist.

As a result of this project, I am more careful about:

- treating user input as data, not code
- checking ownership as well as login
- keeping escaping as the default
- never storing passwords in plaintext, even in a small local app

What I would do differently

If I repeated the project, I would:

- write a short background section earlier, while implementing, instead of leaving research until the end

- add a few automated tests for the fixed version, especially ownership checks and escaped rendering
- record one continuous demo video covering vulnerable and fixed behaviour for each issue

Those changes would not change the core result, but they would make the evidence cleaner.

6. Conclusion

VulnerableNotes achieved its goal. I built a local notes app, introduced four common web vulnerabilities on purpose, demonstrated each one safely, and patched them in SecureNotes.

The main outcome is not just that the exploits worked and the fixes blocked them. The main outcome is that I can now point to specific lines of code and explain why they were unsafe, what impact they had, and what secure alternative replaces them.

For COMP6841, this project was a practical way to connect course concepts to implementation. The vulnerabilities were ordinary. The fixes were ordinary too. That is exactly why they matter.

7. References

- [1] OWASP Foundation, “OWASP Top Ten,” <https://owasp.org/www-project-top-ten/>
- [2] OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- [3] OWASP Foundation, “Cross Site Scripting Prevention Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- [4] OWASP Foundation, “Authorization Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- [5] OWASP Foundation, “Password Storage Cheat Sheet,” https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- [6] Pallets Projects, “Flask Documentation,” <https://flask.palletsprojects.com/>
- [7] Pallets Projects, “Werkzeug Security Helpers,” <https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>

8. Appendix

A. How to run the apps

Vulnerable version:

```
source .venv/bin/activate
flask --app vulnerable_version/app.py init-db
python3 vulnerable_version/app.py
# http://127.0.0.1:5000/
```

Fixed version:

```
source .venv/bin/activate
flask --app fixed_version/app.py init-db
python3 fixed_version/app.py
# http://127.0.0.1:5001/
```

B. Screenshot index

Screenshots are stored under `docs/screenshots/`:

- `milestone_01_scaffold/` — initial homepage
- `milestone_02_auth/` — register and login
- `milestone_03_notes/` — create, view, and search notes
- `milestone_04_sql_injection/` — normal search vs injection result
- `milestone_05_stored_xss/` — payload entry and rendered XSS effect

- `milestone_06_idor/` — Bob viewing Alice's note by changed URL
- `milestone_07_passwords/` — registration and plaintext password evidence
- `milestone_08_fixed_version/` — SecureNotes homepage and blocked exploit results

C. Vulnerability writeups

Full writeups:

- `docs/vulnerability_writeups/sql_injection.md`
- `docs/vulnerability_writeups/stored_xss.md`
- `docs/vulnerability_writeups/idor.md`
- `docs/vulnerability_writeups/weak_password_handling.md`

D. Key code comparisons

Vulnerable search (string-built SQL): see `vulnerable_version/app.py, search()`

Fixed search (parameterized SQL): see `fixed_version/app.py, search()`

Vulnerable note body rendering:

```
{{ note["body"] | safe }}
```

Fixed note body rendering:

```
{{ note["body"] }}
```

Vulnerable note lookup: `WHERE id = ?`

Fixed note lookup: `WHERE id = ? AND user_id = ?`

Vulnerable password storage: plaintext insert and equality check

Fixed password storage: `generate_password_hash()` / `check_password_hash()`

E. Progress evidence

Development progress is recorded in `docs/progress_log.md`. That log shows the milestone-by-milestone build process and supports the claim that this project involved sustained implementation, testing, and documentation work.